

Cómo traducir el enunciado en el esqueleto de la aplicación

Esta guía no entrega la solución — entrega **preguntas y señales de lectura** para que los propios estudiantes lleguen a decidir qué archivos, clases, `sealed class` y `data class` necesitan, a partir del enunciado de FastGo Delivery.

La idea es que antes de escribir una sola línea de Kotlin, el estudiante complete un "mapa de diseño" en papel o pizarra, usando el enunciado como única fuente. Esta guía está organizada como una dinámica de lectura dirigida, requerimiento por requerimiento.

Paso 0 — Estrategia general de lectura

Antes de repartir el caso de negocio, dales esta técnica de lectura:

1. **Subraya los sustantivos que se repiten** en el glosario y en los requerimientos (ej. "pedido", "repartidor", "cliente", "comprobante"). Cada sustantivo que tiene *varios datos asociados* es candidato a ser una clase.
2. **Busca listas cerradas de opciones** ("puede estar en uno de los cuatro estados siguientes", "reconoce tres tipos de repartidor"). Un conjunto cerrado y conocido de opciones es candidato a `sealed class` (si alguna opción necesita datos propios) o a una jerarquía de clases con herencia (si cada opción se comporta distinto).
3. **Busca frases que indiquen "cada uno hace algo distinto"** ("cada uno determina el monto de manera diferente"). Esa es la señal más directa de que se necesita **herencia y polimorfismo**, no un solo `if/else` gigante.
4. **Busca palabras de tiempo y espera** ("no es instantánea", "toma tiempo", "no debe bloquearse", "simula la comunicación y dura X segundos"). Esa es la señal de que se necesitan **corrutinas** (`suspend fun`, `delay`).
5. **Busca la palabra "debe continuar funcionando" o "sin interrumpir"** junto a una lista de errores posibles. Esa es la señal de que se necesita **manejo de excepciones** (`try-catch`), no dejar que el programa se caiga.
6. **Busca preguntas de negocio** ("¿cuántos...?", "¿cuáles son...?", "¿qué pedido...?"). Cada pregunta de este tipo generalmente se resuelve con una función sobre una colección (`filter`, `map`, `count`, `sumOf`, `maxOrNull`).

Esta lectura, hecha en conjunto la primera vez, es más valiosa que darles la lista de clases directamente — es exactamente lo que van a tener que hacer solos en la evaluación real.

Paso 1 — Identificar las entidades (candidatas a clase) leyendo el glosario

Lean el glosario y respondan: **¿de qué "cosas" habla este sistema, y qué datos tiene cada una?**

Término del glosario	Pista	Pregunta para el curso
"Pedido: unidad que representa una solicitud de entrega. Tiene un código único y un estado."	Sustantivo con datos propios	¿Qué datos necesita cada pedido para existir? (→ ver R1)
"Repartidor: recurso... cada repartidor tiene un tipo de vehículo y un estado."	Sustantivo con datos propios	¿Qué datos necesita cada repartidor? ¿Cuántos hay en el sistema? (→ ver R2 y R3)
"Comprobante: documento emitido al completarse la entrega."	Sustantivo que se crea en un momento puntual	¿Cuándo se crea? ¿Qué datos guarda? (→ ver R4)
"Cliente nuevo / frecuente / VIP"	Categorías con una regla asociada (el descuento)	¿Es una entidad con datos propios, o solo una categoría con un porcentaje? (discutir alternativas, ver Paso 5)

Todavía no sé si se resuelve con `class`, `data class` o `sealed class` — solo con "esto es una entidad que el sistema necesita representar". Eso viene después, cuando entiendas el comportamiento de cada una.

Paso 2 — De R1: herencia y polimorfismo (no un `if` gigante)

R1:

"El sistema debe reconocer tres tipos de repartidor... Los tres tipos comparten un conjunto de datos comunes, pero cada uno determina el monto a cobrar de manera diferente."

Preguntas guía:

- ¿Qué datos son **comunes** a los tres tipos? (Respuesta esperada: código, distancia, fecha de creación, tipo de cliente — están listados explícitamente como "datos comunes a todos los pedidos".)
- ¿Qué es **distinto** en cada tipo? (La tarifa base y la regla de recargo/mínimo.)
- Si tuvieran que escribir una función `calcularCosto()`, ¿la escribirían una sola vez con un `if (tipo == "bicicleta")... else if...`, o de otra forma?

Este es el momento debería pensar en `open class` + herencia: una clase base con los datos comunes, y una subclase por cada tipo que **sobrescribe** solo el cálculo que le corresponde. La pista textual clave es "comparten... pero cada uno determina... de manera diferente" — eso es, casi literalmente, la definición de polimorfismo.

Paso 3 — De R2: estados con datos propios → `sealed class`

R2. Preguntas guía:

- ¿Cuántos estados posibles tiene un repartidor? (Cuatro: Disponible, Ocupado, Procesando, Fuera de servicio — un número **cerrado y conocido**.)
- ¿Alguno de esos estados necesita guardar información adicional? (Sí: "Ocupado... el sistema debe saber qué pedido es"; "Procesando... el sistema debe indicar el motivo"; "Fuera de servicio... debe registrar el motivo".)
- Si usaran un simple texto (`estado: String`) para representarlo, ¿qué problema tendrían para guardar "qué pedido" o "qué motivo" junto al estado?

Esta última pregunta es la clave: un `String` no tiene dónde guardar un dato adicional de forma segura, y un `enum` tradicional tampoco (sin trabajo extra). Ahí es donde se introduce `sealed class`: cada estado es un caso, y los que necesitan datos extra los llevan en su propio constructor (`data class Ocupado(val pedido: Pedido)`).

Pregúntate también: *"si más adelante alguien agrega un quinto estado, ¿cómo se darían cuenta de todos los lugares del código que tienen que actualizar?"* — esto conecta con la ventaja real de `sealed class` + `when`: el compilador obliga a cubrir todos los casos.

Paso 4 — De R3 y R4: dónde va cada validación, y qué preguntas se resuelven con colecciones

R3 mezcla dos cosas distintas que conviene separar en la conversación:

1. El **orden del cálculo** (costo → IVA → descuento VIP). Pregúntales: *"si el descuento VIP se aplicara antes del IVA en vez de después, ¿el resultado sería el mismo?"* (no lo es) — esto te obliga a leer el orden con cuidado, no a asumirlo.
2. Las **reglas de validación** (código inválido, monto ≤ 0 , tipo de cliente inválido). Pregúntate: *"¿esto se valida cuando se crea el pedido, cuando se calcula el monto, o en ambos momentos?"*

R4 es una lista de preguntas de negocio. Por cada una, reescribelas como una operación sobre una lista:

Pregunta del enunciado	Operación sobre una colección
¿Cuántos repartidores están disponibles?	contar los que cumplen una condición
¿Qué pedidos son de clientes VIP?	filtrar por una condición
¿Cuál es el ingreso promedio?	sumar y dividir por la cantidad
¿Cuáles son los códigos de los pedidos entregados?	transformar una lista de pedidos en una lista de códigos
¿Qué pedido recorrió más distancia?	encontrar el máximo según un criterio

No hace falta nombrarles `filter/map/sumOf/maxByOrNull` de inmediato — mejor que encuentres el patrón ("varias de estas preguntas se resuelven recorriendo una lista de una forma parecida") y luego recuerda que Kotlin tiene una función para cada patrón, en vez de escribir un `for` con un acumulador cada vez.

Paso 5 — El caso del Cliente: una discusión de diseño abierta (no hay una sola respuesta correcta)

Vale la pena dedicarle un momento aparte, porque es donde más dudan. El enunciado dice que hay tres tipos de cliente con un descuento fijo cada uno. Pregúntate:

- ¿Es necesario que `Cliente` sea una clase separada, o podría ser solo un campo dentro de `Pedido`?

- Si fuera una clase separada, ¿qué datos necesita? (tipo, porcentaje de descuento)
- ¿Se comporta distinto según el tipo (como los repartidores), o solo cambia un número (el descuento)?

Esta última pregunta es la que determina la solución: como el cliente **no tiene comportamiento distinto** (no hay un `calcularAlgo()` que cambie según el tipo), no necesita herencia ni polimorfismo — alcanza con una clase simple que **transporte** el tipo y el descuento. Ahí se introduce `data class` como la herramienta para "clases que son solo datos".

Hay más de una forma válida de resolverlo (un `enum`, una `data class` con `companion object` que valide el texto ingresado, o incluso una `sealed class` si quisieran forzar el descuento a un valor fijo por tipo) . Lo importante es que entiendas *por qué* el cliente no necesita el mismo tratamiento que el repartidor.

Paso 6 — De R5: dónde aparecen las corrutinas

R5. Las frases que hablan de tiempo: "no es instantánea", "toma tiempo", "no debe bloquearse", "dura 2,5 segundos", "dura 5 segundos". Pregúntate:

- ¿Qué pasaría si esa espera se hiciera con `Thread.sleep()` en vez de una corrutina?
- ¿En qué momento exacto de la operación de asignación cambia el estado del repartidor? (Antes de esperar: Procesando. Después de esperar: Ocupado.)

Esto conecta directamente el Paso 3 (`sealed class` de estados) con las corrutinas: la función de asignación **cambia el estado dos veces**, con una espera (`delay`) en el medio.

Paso 7 — De R6: por qué el manejo de errores no es opcional

R6. Para cada uno, identifica **en qué función del sistema podría ocurrir** (¿al crear el pedido? ¿al buscar un repartidor? ¿al calcular el monto? ¿al buscar el pedido por código?). Esto ayuda a que no traten el manejo de errores como un bloque `try-catch`

genérico al final, sino como algo que se decide **función por función**, en el lugar exacto donde el dato podría fallar.

Paso 8 — De R7: cómo repartir todo esto en archivos

Una vez que ya identificas las entidades, los estados y las reglas, la pregunta de organización de archivos se vuelve casi automática. Pregúntate:

- ¿Qué archivos tendría sentido crear, dado todo lo que acabamos de identificar?
- ¿Qué pasaría si pusieramos todo en un solo archivo `Main.kt`? (Volver a leer la condición: "un proyecto de un solo archivo no permite evidenciar modularidad".)

Una organización razonable a la que deberían poder llegar solos:

- Un archivo para el/los estado(s) (`sealed class`).
- Un archivo para la jerarquía de pedidos (`open class` + subclases).
- Un archivo para el resto de las clases de datos y la lógica de negocio.
- Un archivo `Main.kt` que arma los datos de prueba y ejecuta el flujo.

No es necesario que lleguen exactamente a 4 archivos — lo importante es que puedan justificar **por qué** agruparon las cosas como lo hicieron, en vez de copiar una estructura de memoria.

Plantilla para que los estudiantes completen antes de programar

Puedes convertir esto en una hoja de trabajo previa a la ejecución práctica:

1. **Entidades identificadas:** _____
2. **¿Cuál necesita herencia? ¿Por qué?** _____
3. **¿Cuál necesita `sealed class`? ¿Qué estados tiene y cuáles llevan datos extra?** _____

4. **¿Cuál es solo una data class? ¿Por qué no necesita comportamiento propio?** _____
5. **¿Dónde aparecen operaciones asíncronas? ¿Qué cambia de estado antes y después de cada espera?** _____
6. **Lista de errores posibles y en qué función ocurre cada uno:** _____
7. **Archivos que voy a crear y qué va en cada uno:** _____

Si completan esta plantilla razonando a partir del enunciado —y no copiándola de un ejemplo ya resuelto—, ya tienen el esqueleto completo de la aplicación antes de escribir la primera línea de código.